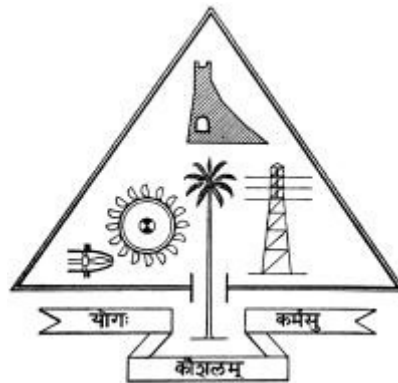


A BEHAVIORAL AND EXPLAINABLE AI FRAMEWORK FOR AUDITING TEXT CLASSIFIERS

*Mini Project Report submitted in partial fulfillment of the requirements for the
award of the degree of **Master of Computer Applications** of the **APJ Abdul
Kalam Technological University***

submitted by

**ZIYA FATHIMA M P
(TCR25MCA-2060)**



**DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE
THRISSUR - 680009**

OCTOBER 2026

ABSTRACT

Modern text classifiers achieve high aggregate accuracy, yet this single metric conceals systematic weaknesses in how models handle specific linguistic phenomena such as negation, double negation, and contrastive connectives. While recent Explainable AI (XAI) research has made model predictions interpretable, explanation quality is typically evaluated on unmodified, real-world input judged by human perception of trust and clarity; no existing framework systematically tests whether a model’s prediction, and the explanation behind it, remain consistent when the input is deliberately and controllably perturbed.

To address this gap, we propose a lightweight, model-agnostic diagnostic framework requiring no access to model internals, training data, or gradients, operating on a strict “pretrained model in, diagnostic report out” principle. The pipeline loads a target black-box classifier from the HuggingFace Hub, then uses spaCy, NLTK WordNet, and rule-based tables to automatically generate controlled linguistic variants from a single input sentence, measuring prediction flip rates and Expected Calibration Error (ECE). For every flipped or unexpectedly stable prediction, the framework extracts local token attributions using LIME and SHAP, comparing explanations between the original and perturbed inputs to classify failures into a three-way taxonomy: *Blind* (the model ignores negation), *Spurious* (the model over-relies on unrelated entities), and *Misweighted* (the model assigns wrong token balance). The pipeline culminates in a structured Solution Report detailing failure counts, empirical examples, and actionable recommendations

CONTENTS

1	INTRODUCTION	1
1.1	Objective	3
2	LITERATURE REVIEW	5
3	ENVIRONMENTAL STUDY	10
3.1	System Configuration	10
3.1.1	Hardware Requirements	11
3.1.2	Software Requirements	11
3.1.3	Software Requirements for Development	12
3.2	Software Specification	13
3.2.1	Python 3.11	13
3.2.2	Transformers and Hugging Face Hub	14
3.2.3	spaCy and NLTK	14
3.2.4	pandas and NumPy	14
3.2.5	LIME and SHAP	15
3.2.6	Matplotlib	15
3.2.7	Python logging	15
3.2.8	Visual Studio Code	16
3.2.9	Git	16
3.2.10	pytest	16
3.2.11	Jupyter Notebook	17
3.2.12	MkDocs	17
4	SYSTEM ANALYSIS	18
4.1	Requirements Analysis	18
4.2	Existing System	19
4.3	Limitations of Existing System	20
4.4	Proposed System	20
4.5	Advantages of Proposed System	21
4.6	Functional Requirements	22
4.6.1	User Interface	22
4.6.2	Perturbation Generation	22
4.6.3	Behavioral Testing	22
4.6.4	Explainability Analysis	23
4.6.5	Report Generation	23
4.7	Non-Functional Requirements	24
4.7.1	Scalability	24
4.7.2	Usability	24
4.7.3	Reliability	24
4.7.4	Maintainability	25
4.8	Feasibility Study	25
4.8.1	Technical Feasibility	25

4.8.2	Operational Feasibility	25
4.8.3	Economical Feasibility	26
	Bibliography	28

CHAPTER 1

INTRODUCTION

Natural Language Processing (NLP) has become one of the most widely deployed branches of Artificial Intelligence (AI), playing an important role in many real-world applications. Text classification models are increasingly used to determine the sentiment of customer reviews, categorize support requests, filter online content and assist decision-making across various industries. Among these applications, sentiment analysis has emerged as one of the most popular techniques for understanding public opinion from large volumes of textual data. Since manually analyzing thousands of reviews or comments is impractical, organizations increasingly depend on automated text classification systems. As these systems move from research environments to real-world deployment, ensuring that their predictions are reliable and trustworthy becomes an essential requirement.

Although modern text classification models achieve high accuracy on standard benchmark datasets, their reliability cannot be judged solely by aggregate performance measures. A single accuracy score summarizes the overall performance of a model but often conceals situations where the model consistently fails on specific categories of inputs. For example, a classifier may correctly identify the sentence “The movie was great” as positive but fail to recognize the change in sentiment when the sentence becomes “The movie was not great.” Similar challenges arise with double negation, contrastive connectives such as “however” or “but,” and other linguistic structures that require understanding the relationship between words rather than treating them independently. These common language patterns demonstrate that

traditional evaluation metrics provide only a partial understanding of model behaviour.

The increasing complexity of modern language models has further intensified this challenge. Earlier machine learning classifiers often consisted of comparatively simple algorithms whose decision-making process could be analyzed directly. In contrast, current state-of-the-art systems are based on large pre-trained transformer architectures that are fine-tuned for specific downstream tasks. Most publicly available models are distributed as ready-to-use systems, allowing users to obtain predictions without access to the model's internal reasoning, training data or optimization process. Consequently, these models function as black boxes, where users can only observe the relationship between the given input and the generated prediction. Under such circumstances, understanding the strengths and weaknesses of a model requires systematic evaluation rather than casual observation.

Explainable Artificial Intelligence (XAI) has emerged as an important research area to improve the transparency of machine learning models. Several studies have proposed techniques that help users understand why a model produces a particular prediction by highlighting influential words or assigning importance scores to different input features. Widely adopted explanation methods such as Local Interpretable Model-Agnostic Explanations (LIME) and SHapley Additive exPlanations (SHAP) have been successfully applied to sentiment analysis, malware detection, fake review identification and various other domains. These techniques improve human understanding of individual model predictions and contribute towards building greater trust in AI systems. However, most existing studies evaluate explanations generated from a single input instance and focus primarily on how users perceive those explanations.

While explainability research focuses on interpreting individual predictions, robustness research investigates how model predictions change when the input is modified. Existing robustness evaluation methods can determine whether a classifier changes its prediction after

introducing controlled perturbations such as negation or word substitution. However, these approaches generally measure only the prediction outcome without examining whether the reasoning behind the prediction has also changed appropriately. Similarly, explainability methods provide detailed interpretations for individual predictions but do not compare how explanations evolve when the meaning of the input sentence changes. Consequently, behaviour analysis and explanation analysis remain two independent evaluation processes, leaving an important gap in understanding the true reliability of modern text classifiers.

Motivated by this limitation, this project proposes **Mapping the Blind Spots: A Behavioral and Explainable-AI Framework for Auditing Text Classifiers**. Instead of developing a new text classification model, the proposed framework evaluates existing pre-trained classifiers by treating them as black-box systems. The framework automatically generates controlled linguistic variations of an input sentence, including negation, double negation, connective insertion and word substitution. Each generated sentence is evaluated using the target classifier, and whenever unexpected prediction changes occur, local explanations are generated for both the original and modified inputs. By comparing these explanations, the framework identifies whether the model ignored the modified words, assigned incorrect importance to them or produced a correct prediction using unrelated reasoning.

1.1 Objective

The objectives of this project are as follows:

- To design a black-box auditing framework capable of evaluating pretrained text classifiers without requiring retraining, model modification, or access to internal parameters.
- To automatically generate controlled linguistic perturbations, including negation, double negation, connective/contrast insertion, and word substitution, from a given input sentence.

- To evaluate the target classifier’s prediction behaviour under these perturbations by measuring prediction flip rates and related confidence changes.
- To generate local explanations using LIME and SHAP for both the original and perturbed inputs, and to compare them for consistency.
- To classify identified failures into a three-way taxonomy—Blind, Spurious, and Misweighted—based on the relationship between prediction outcomes and explanation shifts.
- To produce a structured Model Behavior Report summarizing failure counts, perturbation-to-failure patterns, representative examples, and actionable recommendations.
- To develop a lightweight, reusable, and model-agnostic auditing methodology applicable to any publicly available pretrained text classifier.

As the adoption of large language models and transformer-based classifiers continues to grow, ensuring both reliable predictions and trustworthy explanations has become increasingly important. Understanding the current state of research in explainable AI, behavioural testing and robustness evaluation is therefore essential before developing such an auditing framework. Consequently, the following chapter presents a detailed review of the existing literature on explainable AI techniques, behavioural testing methodologies, robustness analysis and model interpretation approaches, thereby identifying the research gap addressed by the proposed framework.

CHAPTER 2

LITERATURE REVIEW

Deep learning models for text classification, such as sentiment analysis, are now very accurate. But this accuracy hides a real problem that these models are opaque. We often cannot tell why a model made a certain prediction, or whether that prediction would survive even a small change in the input. A single flipped label may reflect genuine understanding of the change, or it may reflect a completely wrong reason. Aggregate accuracy alone cannot tell these two cases apart. This has drawn growing attention from two directions: behavioral testing, which probes whether a model's predictions hold up under controlled linguistic change, and explainable AI (XAI), which asks whether a model's stated reasoning can be trusted.

The earliest and most influential behavioral testing framework for NLP is CheckList. **Ribeiro *et al.*** [1] introduced CheckList, which organizes tests around linguistic capabilities, such as negation, vocabulary, and named entities, and three test types: simple functionality tests, invariance tests, where the prediction should not change, and directional tests, where it should. Applying CheckList to commercial sentiment and question-answering systems, the authors found serious hidden failures, especially around negation, even in models that scored well on standard benchmarks. They also showed that testers using CheckList found about three times more bugs than testers without it. CheckList proved that accuracy alone is not enough to trust a model, but it only checks whether the output label changes. It gives no insight into the reasoning behind a failure.

One limitation of CheckList is that its perturbations are hand-written and

task-specific, which makes it hard to scale. **Dhole *et al.*** [2] addressed this with NL-Augmenter, a large, community-built library of 117 text transformations and 23 filters. Each transformation is tagged with information about what kind of change it makes and how it is expected to affect the output. Testing these transformations across several models, the authors found that character-level and morphological changes hurt accuracy far more than simple word-level changes, sometimes by over 30 percentage points. NL-Augmenter is a good source of reusable, well-documented perturbations, and its tagging system is a useful model for building a failure taxonomy. However, like CheckList, it only reports accuracy drops. It never looks inside the model to see which words the model was actually relying on.

A separate line of work turned to negation specifically, and to how poorly it is handled beyond classification tasks. **Anschütz *et al.*** [3] show that even automatic evaluation metrics for text generation, such as BERTScore and BLEURT, fail to properly penalize negation. A sentence that has been negated, changing its meaning completely, can still receive a high similarity score from these metrics. To fix this, the authors build a rule-based negator, create a new evaluation dataset called CANNOT, and fine-tune a negation-aware metric called NegBLEURT. Their results confirm that negation is not just a classification problem. It is a broader weakness across many NLP systems built on transformer models.

Diwali *et al.* [4] survey the full landscape of XAI methods used in sentiment analysis, including LIME, SHAP, and attention-based visualization. They note that despite growing interest in explainability, evaluation methods for these explanations are still immature, and mostly rely on how convincing an explanation looks to a person, rather than on any behavioral or systematic test. This survey is useful for understanding the field as a whole, but it does not run any new experiments, and so it does not close the gap it identifies.

Returning to negation, **Ilmawan *et al.*** [5] focus more narrowly on how sentiment analysis systems handle it. Their review compares rule-based, lexicon-based, and machine-learning approaches to negation detection, and

finds that no single method performs well across all cases. Simple negation, like “not good,” is usually handled fine. But more complex cases, such as double negation or negation placed far from the sentiment word, still cause frequent errors. This paper reinforces, from a purely sentiment-analysis angle, that negation remains an unsolved problem, and it deserves special attention in any robustness testing pipeline.

While the studies above test predictions, a separate line of research asks whether we can trust the explanations models give for those predictions. **Schoenegger *et al.*** [6] directly compare SHAP, LIME, and Anchor explanations for detectors of machine-generated text. They test the explanations on three properties: faithfulness, whether the explanation matches what the model actually used, stability, whether it stays consistent under small changes, and usefulness to humans. Their key finding is striking: LIME was rated the most trustworthy and useful by human participants, but it was also the method that most hurt participants’ ability to predict what the model would do next. In other words, an explanation can look convincing while being unreliable.

Mysliwiec *et al.* [7] proposed AI-Q, a Python framework that loads a model directly from HuggingFace, applies a set of perturbations to input text, and measures whether the model’s prediction changes. It defines a single robustness score, R , as the fraction of perturbed inputs where the prediction stays the same. The perturbations are grouped into six types, including typos, entity replacement, and negation. Across nine models and five tasks, the authors find that sentiment and emotion models are especially weak to negation perturbations, with robustness dropping by up to 27 percentage points. This is a strong and useful result, but AI-Q stops at the prediction level. It tells us that a prediction flipped, but not why. A model that flips because it correctly noticed the negation, and a model that flips for the wrong reason entirely, look identical under the R score.

Abhiram *et al.* [8] take a more human-centered approach to explanation evaluation. They apply LIME and SHAP to a sentiment classifier trained on

Amazon reviews, and run a usability study with 123 participants to measure how clear, relevant, and trustworthy people find the explanations. They find that trust and clarity improve as people see more explanations, and that LIME and SHAP tend to highlight similar important words. This is a useful study, but all of it happens on normal, unmodified review text. The explanations are never tested against a slightly changed version of the same sentence, so there is no way to know if the explanation would still make sense if the input changed even a little.

Across these studies, a consistent pattern emerges. Behavioral testing frameworks are effective at detecting when a model's prediction changes under perturbation, and they consistently point to negation as a major weak spot, yet none of them look at why the prediction changed, since none of them generate or compare explanations. Independent studies on negation confirm that it remains one of the most poorly handled phenomena in NLP, well beyond ordinary edge cases. Meanwhile, work on explanation methods such as LIME and SHAP shows that these methods can be evaluated for human trust and clarity, but almost always on static, unperturbed inputs. One study comes closest to bridging the two directions, showing that human trust and explanation faithfulness can directly diverge, but its perturbation strategy is built specifically for a single narrow task and does not generalize. Taken as a whole, the reviewed literature treats prediction robustness and explanation quality as separate questions, tested on separate inputs, evaluated by separate methods. A model's prediction is stress-tested under perturbation, or its explanation is judged for clarity and trust, but rarely both at once, and never by comparing the same explanation before and after a controlled linguistic change. This leaves a quiet but consequential blind spot: a classifier can pass every accuracy benchmark and still be relying on the wrong words, and an explanation can look convincing to a human reader while telling a different story than the one the model actually followed. The present work addresses this directly by pairing perturbation-based testing with paired explanation comparison, applying LIME and SHAP not to a single static input but to

matched original-perturbed sentence pairs. Instead of asking only whether a prediction flipped, or only whether an explanation looks convincing, it asks whether the model's own reasoning stays consistent when the input changes. The resulting shift in attribution, together with prediction flip rate and calibration error, is used to sort failures into a reusable taxonomy: Blind, Spurious, and Misweighted, giving a concrete account of why a model fails, not just that it fails.

CHAPTER 3

ENVIRONMENTAL STUDY

This chapter presents the environmental settings required for the successful development and evaluation of the Behavioral and Explainable-AI Framework for Auditing Text Classifiers. The environment ensures that the system can load pretrained models, generate input perturbations, test and measure performance, diagnose failures and finally produce human-readable reports in a reliable and efficient manner.

The environment includes both hardware and software prerequisites that allow all stages of the diagnostic pipeline to work together seamlessly. It covers the tools, libraries and platforms used from model loading through perturbation generation and evaluation, up to report preparation and recommendation generation.

The proposed framework is developed entirely using free and open-source software technologies. The use of open-source tools ensures reproducibility, minimizes deployment cost and enables the framework to be extended without dependency on proprietary platforms or commercial services.

3.1 System Configuration

System configuration refers to the collection of hardware and software components needed to implement and run each step of the diagnostic workflow. Proper configuration is essential to handle model inference, data processing and explanation methods efficiently. It also ensures that the modules for perturbation generation, metric computation and failure analysis can communicate correctly throughout the pipeline.

The main operations carried out by the system include loading pretrained

models, generating controlled perturbations, running inference, computing metrics such as flip rate and calibration error, performing attribution-based diagnosis and compiling reports. Therefore, a carefully chosen configuration is crucial to reduce processing time and avoid bottlenecks during experimentation and evaluation.

3.1.1 Hardware Requirements

The hardware requirements specify the minimum resources needed to run the end-to-end diagnostic pipeline, including inference for pretrained models and the analysis of large sets of perturbed examples.

The key hardware requirements are detailed as follows:

- **Processor:** Intel® Core™ i5 Processor / AMD Ryzen™ 5 Processor or higher, to support model inference and data processing tasks.
- **Memory:** Minimum 8 GB RAM, to handle batches of perturbed inputs and store intermediate analysis results without performance degradation.
- **Storage:** 20–50 GB of free disk space, for storing datasets, perturbation outputs, diagnostic summaries and generated reports.
- **Network:** Stable Internet connectivity for downloading pretrained models and dependencies from public repositories.

3.1.2 Software Requirements

The software requirements define the operating systems, programming language and core libraries used to support model loading, perturbation generation and diagnostic evaluation. These tools ensure compatibility between all modules of the framework and enable smooth integration with model repositories and explanation techniques.

The key software requirements are detailed as follows:

- **Operating System:** Ubuntu 22.04 LTS, Windows 10/11 or macOS, supporting cross-platform development and execution.

- **Programming Language:** Python 3.11 or higher, serving as the primary language for implementing all components of the framework.
- **Version Control System:** Git, for managing source code, experiment scripts and configuration files.
- **Required Python Libraries:**
 - Transformers and Hugging Face Hub – for loading pretrained text classification models and tokenizers from public repositories.
 - spaCy and NLTK – for linguistic preprocessing and controlled perturbation generation.
 - pandas and NumPy – for organizing datasets, computing metrics and summarizing experimental results.
 - LIME and SHAP – for generating local explanations and performing attribution-based diagnosis.
 - Matplotlib – for producing graphs and visualizations of evaluation results.
 - Python logging – for recording experimental activities and diagnostic information.

3.1.3 Software Requirements for Development

In addition to execution-time software, several development tools are required to design, test and maintain the diagnostic pipeline. These tools assist in writing and debugging code, managing experiments and documenting the framework.

The key development software requirements are detailed as follows:

- **Operating System:** Ubuntu 22.04 LTS, Windows 10/11 or macOS, enabling consistent development across platforms.
- **Back End:** Python 3.11 or later, used for implementing all stages of the workflow.

- **IDE Used:** VSCodium or Visual Studio Code, providing support for Python development, debugging and extension-based tooling.
- **Version Control:** Git, with a remote repository used for source-code backup and change tracking.
- **Testing Tools:** pytest, for verifying the correctness of individual modules of the framework.
- **Documentation Tools:** MkDocs, for authoring and generating technical documentation.
- **Visualization:** Graphviz and Matplotlib, for generating workflow diagrams and experimental figures.
- **Notebook Environment:** Jupyter Notebook, for exploratory experimentation and result analysis.

3.2 *Software Specification*

The following section provides a detailed description of the software components and technologies used to develop the Behavioral and Explainable-AI Framework for Auditing Text Classifiers. Each component performs a specific task related to model loading, perturbation generation, diagnostic analysis and report generation.

3.2.1 *Python 3.11*

Python is the primary programming language used to implement the proposed framework. Its extensive ecosystem of machine learning, natural language processing and scientific computing libraries allows model loading, perturbation generation, explanation computation and report generation to be implemented within a single and consistent development environment.

Python was preferred over compiled languages such as Java and C++ because the majority of modern machine learning and explainable AI frameworks, including Transformers, spaCy, LIME and SHAP, are

primarily developed and maintained for Python. This significantly reduces implementation complexity and ensures compatibility with widely used research libraries.

3.2.2 *Transformers and Hugging Face Hub*

The Transformers library and Hugging Face Hub are used to load pretrained text classification models and their associated tokenizers directly from publicly available repositories. These tools provide a unified interface for model loading and inference, allowing different transformer architectures to be evaluated without retraining.

The Hugging Face ecosystem was selected because it provides access to a large collection of state-of-the-art pretrained models through a standardized interface. This flexibility is essential for a framework designed to evaluate multiple black-box classifiers under identical testing conditions.

3.2.3 *spaCy and NLTK*

spaCy and NLTK are employed to perform linguistic preprocessing and generate controlled perturbations of input sentences. These libraries provide lexical resources, tokenization capabilities and language-processing utilities that enable the system to introduce negations, perform synonym substitutions and apply pattern-based transformations.

spaCy and NLTK were selected because they complement one another and provide mature implementations of many natural language processing tasks required by the framework. Their extensive documentation and active communities make them suitable choices for academic research and experimental evaluation.

3.2.4 *pandas and NumPy*

pandas and NumPy are used for organizing experimental data, computing metrics and summarizing results generated by the framework. They support efficient manipulation of large tables of inputs, predictions and perturbation

metadata.

These libraries were preferred because they are widely adopted within the scientific computing community and provide highly optimized implementations of numerical operations and data-processing functions. Their integration with the broader Python ecosystem simplifies statistical analysis and report generation.

3.2.5 *LIME and SHAP*

LIME and SHAP are explainable artificial intelligence libraries used to generate local explanations for model predictions. These techniques identify the importance of individual input features and provide insights into the reasoning process of black-box text classifiers.

LIME and SHAP were selected because they are among the most widely used model-agnostic explanation methods in explainable AI research. Their ability to generate interpretable feature attributions makes them suitable for diagnosing failure cases and identifying blind spots in model behaviour.

3.2.6 *Matplotlib*

Matplotlib is the visualization library used to generate graphs and figures representing experimental findings. It is used to visualize prediction flip rates, explanation consistency scores and other diagnostic metrics produced by the framework.

Matplotlib was preferred because it provides extensive customization capabilities and integrates naturally with the scientific Python ecosystem. The library enables high-quality visualizations suitable for inclusion in reports and academic publications.

3.2.7 *Python logging*

The Python logging module is used to record information concerning model evaluations, generated perturbations and detected failures. It provides structured logging capabilities that assist in debugging and experimental

reproducibility.

The standard logging module was preferred over third-party alternatives because it offers sufficient configurability for the requirements of the framework while avoiding additional dependencies.

3.2.8 Visual Studio Code

Visual Studio Code is the Integrated Development Environment used throughout the development and maintenance of the diagnostic framework. It provides syntax highlighting, debugging capabilities, integrated terminals and extension-based support for Python development.

Visual Studio Code was selected because it provides comprehensive development features while remaining lightweight and freely available. Its extensive extension ecosystem and built-in Git integration improve developer productivity throughout the implementation process.

3.2.9 Git

Git is the distributed version control system used to track the development of the framework and manage changes to source code, experiments and documentation.

Git was preferred because its distributed architecture provides complete local history and simplifies collaboration and experiment management while ensuring reproducibility of research artefacts.

3.2.10 pytest

pytest is the testing framework used to verify the correctness of individual modules of the diagnostic pipeline, including perturbation generation, metric computation and explanation routines.

It was selected because of its concise syntax, extensive plugin ecosystem and ease of integration with Python-based projects.

3.2.11 Jupyter Notebook

Jupyter Notebook is used for exploratory experimentation, result analysis and rapid prototyping of diagnostic techniques. It enables code execution, visualization and documentation within a single interactive environment.

Jupyter Notebook was preferred because it supports reproducible experimentation and is widely adopted in machine learning and natural language processing research.

3.2.12 MkDocs

MkDocs is the static site generator used to author and produce the technical documentation accompanying the framework. It generates a navigable documentation website directly from Markdown source files.

Its lightweight nature and integration with Git make it suitable for maintaining technical documentation alongside the source code.

In this chapter, the hardware, software and development environment required for implementing the Behavioral and Explainable-AI Framework for Auditing Text Classifiers have been described in detail. These settings ensure that each stage of the workflow, from model loading to report generation, operates efficiently and coherently. With the execution environment established, the next chapter focuses on the system analysis, requirements and feasibility of the proposed diagnostic framework.

CHAPTER 4

SYSTEM ANALYSIS

System analysis is the process of structurally examining the proposed framework for auditing text classifiers through behavioral testing and explainable artificial intelligence. The main purpose of the analysis is to comprehend the system's functionality, processing flow and interaction in identifying hidden weaknesses in pretrained text classifiers and generating a structured model behavior report automatically.

The analysis includes an examination of how the user provides a single input sentence to the system, after which the system generates controlled linguistic variants such as negation, double negation, contrastive connectives and word substitutions. These variants are passed through a target pretrained text classifier to observe prediction changes relative to the original sentence. For cases where the prediction flips unexpectedly or fails to flip when expected, the system extracts local explanations for both the original and perturbed input and compares them directly. On this basis, the analysis leads to the production of a behavior report in an efficient way and can be maintained and scaled for practical auditing of text classifiers. It also ensures an appropriate design of the system that specifies functional and non-functional requirements efficiently and confirms that the proposed system meets the user requirements while maintaining accuracy, consistency and interpretability.

4.1 Requirements Analysis

The requirements analysis gives a broad functional and user requirements for auditing pretrained text classifiers using behavioral and explainable AI

methods. It is a requirement that this system will be capable of analyzing a given text classifier and producing a diagnostic report that is easily understood by human readers. All the requirements are specified in the most clear way to be precise, measurable and attainable by the system. The main requirements are that the application should be able to accept an input sentence, generate controlled linguistic perturbations, run the classifier on original and modified inputs, integrate local explanation methods such as LIME and SHAP, compare explanations across versions and provide a user interface to display the findings. Also, the system should be able to support different pretrained text classifiers and optimally process different linguistic categories such as negation, double negation, contrastive connectives and word substitutions. These requirements are mandatory to design a reliable, efficient and intuitive system which can audit model blind spots without causing a failure of user expectations and maintaining a level of accuracy and performance.

4.2 Existing System

As of now, text classifier evaluation systems are either based on standard performance testing or use simple explanation tools. In the conventional process, practitioners evaluate classifiers using benchmark datasets and report aggregate accuracy or similar metrics. While this approach allows for quantitative assessment, it requires limited insight into hidden behavioral weaknesses and heavily depends on the selected evaluation set. Explainable AI systems are also employed to reduce the uncertainty of black-box models. These systems generally generate explanations for individual predictions using methods such as LIME or SHAP. However, most of these systems rely on isolated explanations and lack the capability to compare behavior between original and perturbed inputs, limiting their effectiveness in producing thorough and behaviorally grounded diagnostics.

4.3 Limitations of Existing System

Though there is access to evaluation and explanation technology, the existing systems have the following drawbacks:

- Evaluation is usually based on aggregate metrics, which can hide predictable failures on specific linguistic patterns such as negation or contrast.
- Explanation tools often analyze only one unmodified input at a time, which limits their ability to reveal how model reasoning changes when meaning changes.
- The tools themselves may not fully capture whether the model ignored a changed word, attended to it but weighted it incorrectly, or relied on unrelated cues, limiting the generation of insightful and meaningful diagnostics.
- Some methods struggle to represent the relationships between original and perturbed sentences, making it difficult to map out the overall failure structure.
- Many current evaluation methods support only performance scoring or explanation scoring separately, reducing flexibility and applicability in behavioral auditing workflows.
- Automatically generated outputs often lack direct comparison between behavior and explanation, leaving the purpose, function and interactions of model failures unclear.

4.4 Proposed System

The proposed system is based on a behavioral and explainable-AI framework that is meant to produce accurate, structured and up-to-date diagnostics automatically, based on the behavior of a pretrained text classifier. Differently from other systems, this system combines controlled linguistic

perturbation generation with the use of local explanation methods to understand the structure and semantics of model decisions. The system accepts input in the form of a single sentence. Next, the system can use the generated linguistic variants to test the classifier and generate meaningful outputs such as flip-rate measurements, explanation-consistency scores and failure-type classifications.

4.5 Advantages of Proposed System

The proposed system offers the following advantages:

- Minimal manual work is required for auditing, as the process is fully automated. This allows researchers to focus more on analysis rather than spending time writing repetitive test cases.
- Precise understanding of the classifier's behavior is achieved through controlled perturbation analysis, enabling accurate identification of prediction flips and stability issues. The system provides detailed insights into both the decision process and the model's sensitivity to linguistic changes.
- Diagnostics are human-friendly and context-aware, generated with consideration of the purpose and interactions of specific sentence variants. This helps users read and understand model behavior more quickly and efficiently.
- Model-agnostic support increases versatility, especially for practitioners working with publicly available pretrained text classifiers, eliminating the need for access to internals, gradients or training data.
- Efficient handling of different linguistic categories ensures that the system can provide comprehensive diagnostics across negation, double negation, contrast and substitution cases, aiding in the analysis of real-world language phenomena.

- Enhanced understandability and maintainability of model behavior improve collaboration within teams, ensuring that diagnostic reports support long-term model reliability and effective knowledge sharing.

4.6 Functional Requirements

This section describes the core functionalities that the system must provide to meet user requirements. Functional requirements define the specific operations and behaviors of the system when analyzing a text classifier and generating behavioral diagnostics. They outline what the system should do and serve as a foundation for designing and developing its various modules.

4.6.1 User Interface

This section covers how the user interacts with the system and what kind of input it accepts. Through the user interface, the user provides an input sentence for generating the diagnostic report. The interface is designed for easy interaction, requiring no technical knowledge or skills. The system accepts a single sentence or short text snippet from the user. It handles various text forms and supports different linguistic structures within the sentence. All inputs are validated for correctness before the analysis process begins.

4.6.2 Perturbation Generation

This subsection shows the method that is used for internal linguistic analysis. To ensure correct understanding of sentence behavior, input text is transformed through controlled perturbation generation. Correct perturbation results are ensured to identify all the relevant linguistic variants to work on further correctly.

4.6.3 Behavioral Testing

The following section is a description of how meaningful model behaviors are measured. The system passes the original and perturbed sentences through the classifier and records whether the prediction changes or remains

stable. The code is parsed to extract structured behavior such as prediction flips, unchanged outputs and sensitivity to linguistic modifications, along with comparison results.

4.6.4 Explainability Analysis

This subsection describes explanation comparison processes. For analysis, a local explanation method such as LIME or SHAP is applied to both the original and perturbed inputs. To give the most appropriate human-readable diagnostic output, the system considers token attributions, explanation alignment and explanation-consistency during the analysis procedure.

4.6.5 Report Generation

The following section describes the various document formats produced by this system will hence be described in the following paragraphs. These are a Model Behavior Report, failure summaries and explanation comparison results. This means that the output shall be given in such a way that it is concise and clear, making it easier to analyze the document produced.

These are the generated files:

- **model_behavior_report.md** – Provides a detailed description of the project, including the target model, input sentence, perturbation types, flip-rate results, explanation-consistency scores, failure categories and recommendations.
- **failure_summary.md** – Offers an overview of the system behavior, including blind failures, spurious failures and misweighted failures, with corresponding counts and examples.
- **explanation_comparison.md** – Contains explanation-level diagnostics for original and perturbed inputs, detailing token attributions, internal relationships and the observed changes in model reasoning.
- **probes/*.md** – Contains documentation for each generated probe, detailing the original sentence, transformed sentence, prediction

outcome, explanation output and potential issues such as ignored negation or incorrect attention.

4.7 Non-Functional Requirements

The non-functional requirements specify quality attributes for the system. They ensure that the systems output is accurate, consistent and that it functions effectively and remains user-friendly. The non-functional requirements specify the quality attributes for the system to ensure that it is effective, user-friendly and produces the right output and that it emphasizes on scalable, usability and reliability that underpins implementing a robust and practical behavioral auditing solution.

4.7.1 Scalability

This system must be able to work with both a single sentence and larger groups of probes across multiple evaluation runs. The system's efficiency and uniformity are not compromised with the increasing volume of generated variants. The modular design enables multiple perturbations to be simultaneously processed for efficiency and to maintain consistency in performance for large-scale applications.

4.7.2 Usability

The system must provide an easy, straightforward interface to allow users either to upload their target classifier or provide a model reference and input sentence. The interface should be intuitive and self-explanatory, such that the user is able to generate the diagnostic report without the need for any complex configuration, thereby minimizing any learning requirements for new users.

4.7.3 Reliability

Diagnostics that are generated should be accurate and well structured. It should be able to cope with such issues as unsupported text formats

or explanation failures without breaking the workflow. This requires an efficient error-handling mechanism that prevents the system from crashing and provides a consistent, stable and reliable performance.

4.7.4 Maintainability

The system's architecture should allow for its customization and upgrading when changes in classifier types, perturbation rules or explanation methods occur. It is done by using a modular architecture, well-structured code and configurable components that will help the developers customize or upgrade the features with minimum effort, guaranteeing the long-term usability of the auditing system.

4.8 Feasibility Study

This section discusses the feasibility of the proposed system from technical, operational and economic perspectives. The system is considered viable because it uses open-source tools and readily available pretrained models, eliminating licensing costs and ensuring practical, cost-effective implementation.

4.8.1 Technical Feasibility

This subsection evaluates the technical feasibility of implementing the proposed system. It relies on widely used and mature technologies, including Python, pretrained text classifiers, perturbation utilities and explanation methods such as LIME and SHAP, all supported by active and robust communities. The system's moderate computational requirements allow it to be developed and executed efficiently in standard computing environments.

4.8.2 Operational Feasibility

This subsection analyzes the usability of the proposed system. It requires minimal user input and little technical knowledge, as users only need to

specify the target classifier and provide a sentence, while the system handles all processing automatically.

4.8.3 *Economical Feasibility*

This subsection analyzes cost-related aspects. The system uses open-source software and libraries, avoiding any licensing fees and operates on standard hardware, eliminating additional infrastructure costs. This makes it affordable, cost-effective and accessible for both individual users and institutions.

The analysis of the proposed behavioral and explainable-AI framework was carried out by thoroughly examining the system requirements, identifying the limitations of existing evaluation solutions and evaluating the functional features of the proposed system. This analysis helps in understanding how the system is expected to behave under different scenarios and ensures that all user requirements are adequately addressed. Furthermore, it highlights the gaps in current approaches and justifies the need for an intelligent, automated solution. Overall, this comprehensive analysis provides clear insight into the system's behavior and establishes a strong foundation for the system design presented in the following chapter.

BIBLIOGRAPHY

- [1] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 4902–4912, doi: 10.18653/v1/2020.acl-main.442.
- [2] K. D. Dhole, V. Gangal, S. Gehrmann, A. Gupta, Z. Li, S. Mahamood, A. Mahendiran, S. Mille, A. Shrivastava, S. Tan, T. Wu, J. Sohl-Dickstein, J. D. Choi, E. Hovy, O. Dusek, S. Ruder, et al., "NL-Augmenter: A Framework for Task-Sensitive Natural Language Augmentation," *Northern European Journal of Language Technology*, 2023.
- [3] M. Anschütz, D. Miguel Lozano, and G. Groh, "This is not correct! Negation-aware Evaluation of Language Generation Systems," in *Proceedings of the 16th International Natural Language Generation Conference*, Prague, Czechia, 2023.
- [4] A. Diwali, K. Saeedi, K. Dashtipour, M. Gogate, E. Cambria, and A. Hussain, "Sentiment Analysis Meets Explainable Artificial Intelligence: A Survey on Explainable Sentiment Analysis," *IEEE Transactions on Affective Computing*, vol. 15, no. 3, pp. 837–846, 2024, doi: 10.1109/TAFFC.2023.3296373.
- [5] L. B. Ilmawan, Muladi, and D. D. Prasetya, "Negation Handling for Sentiment Analysis Task: Approaches and Performance Analysis," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 14, no. 3, pp. 3382–3393, 2024, doi: 10.11591/ijece.v14i3.pp3382-3393.

- [6] L. Schoenegger, Y. Xia, and B. Roth, “An Evaluation of Explanation Methods for Black-Box Detectors of Machine-Generated Text,” *arXiv preprint arXiv:2408.14252*, 2024.
- [7] K. Mysliwiec, M. Chinea-Ríos, I. Borrego-Obrador, and M. Franco-Salvador, “AI-Q: A Framework to Measure the Robustness of Text Classification Models,” *Procedia Computer Science*, vol. 246, pp. 596–605, 2024, doi: 10.1016/j.procs.2024.09.468.
- [8] Abhiram, C. N. Amritha, R. Gokul, and P. Keerthimon, “A Human-Centric Evaluation of XAI Methods for Sentiment Analysis on Amazon Reviews,” in *Proceedings of the 2025 4th International Conference on Advances in Computing, Communication, Embedded and Secure Systems (ACCESS)*, 2025, pp. 730–734, doi: 10.1109/ACCESS65134.2025.11135705.